

INTRODUCTION — 2 min

[Slide 1 — couverture]

Bonjour à tous. Je vais vous présenter mon projet SAE204 qui porte sur deux choses : la mise en place d'un serveur d'intelligence artificielle local, et le développement d'un outil de détection de plagiat basé sur cette infrastructure.

[Slide 2 — Enjeux et Contexte]

Le point de départ c'est une problématique concrète. Aujourd'hui, quand on veut utiliser une IA pour analyser des documents académiques — des rapports de TP par exemple — le réflexe naturel c'est d'utiliser ChatGPT ou Gemini. Sauf que ces services sont des boîtes noires : on envoie les données sur des serveurs qu'on ne contrôle pas, et dans un contexte académique avec des copies d'élèves, ça pose des problèmes réels de confidentialité et de conformité RGPD.

[Slide 3 — L'approche]

Mon approche a été progressive. J'ai d'abord testé des modèles de langage sur ma propre machine pour comprendre leurs besoins en ressources. Ensuite j'ai configuré le serveur. Et c'est seulement une fois l'infrastructure en place que j'ai développé le programme de détection.

[Slide 4 — La problématique]

Le cahier des charges fixait plusieurs objectifs : déployer une IA locale, légère, rapide, autonome, qui porte un nom — c'est l'assistant Qwen — et concevoir un système de détection de fraude en Python, accessible via une interface web.

L'angle directeur de tout le projet c'est ce que j'appelle le "**local d'abord**" : 100 % des traitements restent sur notre infrastructure, rien ne sort vers l'extérieur par défaut.

CHOIX TECHNOLOGIQUES — 3 min

[Slide 6 — Pourquoi le Local First]

Avant de vous montrer ce que j'ai construit, je veux expliquer pourquoi j'ai retenu ces outils plutôt que d'autres.

[Slide 8 — La Stack Retenue]

La stack finale c'est trois briques principales. **Ollama** qui est le moteur d'inférence — c'est lui qui télécharge et fait tourner les modèles de langage localement, et qui les expose via une API REST sur le port 11434. **OpenWebUI** qui est l'interface de chat, le côté visible pour l'utilisateur — pensez à quelque chose comme ChatGPT mais auto-hébergé. Et **Python avec Flask** qui est le cœur de l'outil de détection de plagiat.

[Slide 10 — Choix & Écarts]

J'ai évalué d'autres solutions avant d'arriver là. **LM Studio** a été écarté dès le départ : mes processeurs Xeon E5-2630 v3 ne supportent pas le jeu d'instructions AVX2 qu'il requiert — incompatibilité matérielle pure. **LiteLLM** a été installé et testé, mais la liaison avec Ollama n'a jamais été stable lors des tests d'intégration, donc abandonné. Et **HuggingFace** pour la vérification externe — j'y reviendrai — a été écarté pour des raisons à la fois techniques et de conformité RGPD.

DÉVELOPPEMENT — 6 min

Bloc A — L'IA locale — 2 min

[Slide 9 — Architecture Réseau]

Parlons d'abord de l'infrastructure. Le serveur c'est un PC custom équipé de **deux processeurs Xeon E5-2630 v3** — 32 threads au total — d'une **RTX 3060 Ti avec 8 Go de VRAM** et de **32 Go de RAM DDR3**. Il tourne sous **Linux Mint 22.3 en mode headless**, c'est-à-dire sans interface graphique, piloté uniquement via le réseau. Il est connecté en filaire au réseau privé de la salle réseaux, sur le sous-réseau 172.17.0.0.

Les postes clients — mon Lenovo et mon MacBook — accèdent aux services directement via ce réseau.

[Slide 11 — L'IA Locale]

Sur ce serveur, Ollama tourne en service d'arrière-plan et charge plusieurs modèles selon les besoins. L'assistant nommé du cahier des charges c'est **Qwen**, accessible via OpenWebUI sur le port 3000. C'est l'interface conversationnelle que les utilisateurs voient.

En bonus — et je le mentionne comme élément d'extensibilité — j'ai validé l'intégration avec **Home Assistant** via un Raspberry Pi. Ollama dispose d'une intégration native dans HAOS, ce qui permet de connecter l'IA à un assistant vocal domotique. Ça démontre que la stack est extensible bien au-delà du simple chat.

Bloc B — Le pipeline de détection — 4 min

[Slide 12 — Le Pipeline Anti-Plagiat]

Passons au cœur du projet : le pipeline de détection de plagiat.

L'idée centrale c'est de ne pas se limiter à comparer du texte mot pour mot. Mon pipeline combine **quatre signaux indépendants** pour être robuste face aux tentatives de contournement classiques.

[Slide 13 — Indicateurs de Verdict]

Le premier signal c'est la **détection lexicale**. Je découpe les textes en séquences de 5 mots consécutifs — on appelle ça du shingling — et je compare les chevauchements entre les deux rapports avec la bibliothèque rapidfuzz. Ça détecte le copier-coller direct.

Le deuxième signal c'est la **détection sémantique**. Là c'est plus sophistiqué : je transforme chaque phrase en vecteur numérique grâce au modèle multilingual-e5-base. Deux phrases avec le même sens auront des vecteurs proches même si les mots sont complètement différents. C'est ce qui permet de détecter la **paraphrase** — ce que le signal lexical rate complètement.

Le troisième signal c'est l'**analyse d'images**. Pour chaque image extraite des PDFs, je calcule un hash perceptuel. C'est une empreinte numérique robuste aux petites modifications — compression, recadrage léger. Si deux images ont un hash proche, c'est la même image réutilisée.

Le quatrième signal c'est le **verdict IA**. Je sou mets les résultats des trois premiers signaux au modèle gemma3:4b via Ollama, qui retourne un verdict structuré en JSON avec un niveau de confiance et une justification en langage naturel.

Ces quatre scores sont agrégés en un score final pondéré — 35 % lexical, 35 % sémantique, 15 % images, 15 % IA. En dessous de 40 % c'est faible, entre 40 et 69 % c'est modéré, au-delà de 70 % c'est élevé.

[Slide 15 — L'Interface de Gestion]

Pour rendre tout ça accessible sans ligne de commande, j'ai développé une **interface web avec Flask** sur le port 5000. L'enseignant dépose ses deux rapports, lance l'analyse, et voit le pipeline traiter chaque étape en temps réel dans un terminal qui défile. À la fin, un rapport PDF est généré et téléchargeable.

Ce terminal live fonctionne grâce aux **Server-Sent Events** — Flask intercepte la sortie du pipeline et pousse chaque ligne vers le navigateur au fil de l'eau, sans que la page ait besoin de se rafraîchir.

[Slide 14 — Le Pivot Stratégique Local]

J'avais initialement prévu une vérification externe via l'API HuggingFace avec le modèle Kimi K2.6. J'ai testé l'intégration — ça fonctionnait — mais j'ai finalement décidé de tout rapatrier en local. La vérification externe est maintenant assurée par **llama3.1:8b** directement sur Ollama. Le pipeline est donc **100 % local**, sans aucune dépendance réseau externe. C'est cohérent avec le principe de confidentialité du projet et ça règle au passage les questions RGPD.

VIDÉO COMMENTÉE — 2 min

[Slide 16 — Démonstration]

Je vais maintenant vous montrer le système en fonctionnement.

→ *Lancer la vidéo*

[Pendant la vidéo — parler sur les images :]

Là on voit l'interface Flask — à gauche la zone de dépôt des rapports, à droite le résumé du pipeline avec les étapes. Je dépose deux rapports PDF d'étudiants...

Le terminal commence à défiler — vous voyez l'extraction du texte, l'étape lexicale, l'étape sémantique...

Le score final s'affiche avec le code couleur — ici 53 % donc niveau modéré, en orange. Et le rapport PDF est généré automatiquement.

BILAN ET LIMITES — 1 min

[Slide 17 — Bilan & Limites Assumées]

Je veux être honnête sur les limites actuelles.

La contrainte principale c'est la **VRAM** — 8 Go c'est juste pour faire tourner simultanément le modèle de texte et le modèle de vision. J'ai été obligé de désactiver LLaVA, qui analysait les images avec une IA, et de le remplacer par le hash perceptuel uniquement. Et sentence-transformers a été forcé sur le CPU pour libérer de la VRAM.

Les drivers NVIDIA doivent aussi être réinstallés pour garantir l'accélération GPU.

Ce sont des limites que j'assume et que j'ai documentées — elles ont généré des décisions techniques concrètes et traçables.

CONCLUSION — 1 min

[Slide 18 — Conclusion & Ouverture]

Pour conclure : j'ai mis en place un serveur IA fonctionnel, autonome et conforme sur le réseau de l'IUT, avec un outil de détection de plagiat multi-signaux accessible via une interface web.

Les axes d'amélioration sont clairs : affiner les seuils de détection sémantique sur de vrais jeux de données, et réintégrer l'analyse vision si le matériel le permet.

[Slide 19 — Questions]

Je vous remercie pour votre attention, je suis disponible pour vos questions.